

ARCHITECTURE REVIEW PRESENTATION

Hostel Buddy

Smart Hostel Complaint Management System

Group 19

Pranjal Tyagi · Dinesh Saini · Ajinkya · Saurabh · Vijay

Department of Computer Science and Engineering · **IIIT Kottayam**



Agenda & Speaker Allocation

20 minutes

#	Section	Diagram	Time	Presenter
1	Project Overview	—	1 min	Pranjal Tyagi
2	System Requirements	Use Case	2 min	
3	System Architecture	Architecture	3 min	Dinesh Saini
4	Module Decomposition	Decomposition	2–3 min	
5	Layered Architecture & Data Flow	Request Flow	3 min	Ajinkya
6	Complaint Lifecycle	State	2 min	Saurabh
7	Database Design	ER	2 min	
8	Security & Key Design Decisions	—	1–2 min	Vijay
9	OOAD Diagrams	Class · Sequence · Activity · State	3 min	

1 Project Overview

1 min

Problem

- Hostel complaints are recorded in a **paper register**
- No way to search, track or report on them
- Students get **no visibility** after reporting an issue

Objective

- Digitise complaint reporting and resolution
- Give students **real-time status tracking**
- Give the administrator a **single dashboard** with analytics
- Store every complaint **permanently and searchably**

Who uses the system

- **Student** — raises and tracks their own complaints
- **Administrator** — reviews all complaints, updates status, views analytics

Two roles only — no separate maintenance-staff role in v1.0

How it works — in one line

A student submits a complaint through a web interface; it is stored as **Pending**, and the administrator advances it through **In Progress** → **Resolved** → **Closed** while the student watches the status change live.

2 System Requirements

2 min

Student Functional Requirements

- Register / Login
- Manage profile
- Raise complaint
- View complaints
- Edit / Delete **Pending** complaint
- Track complaint status
- View remarks and resolution date
- View personal statistics

Administrator Functional Requirements

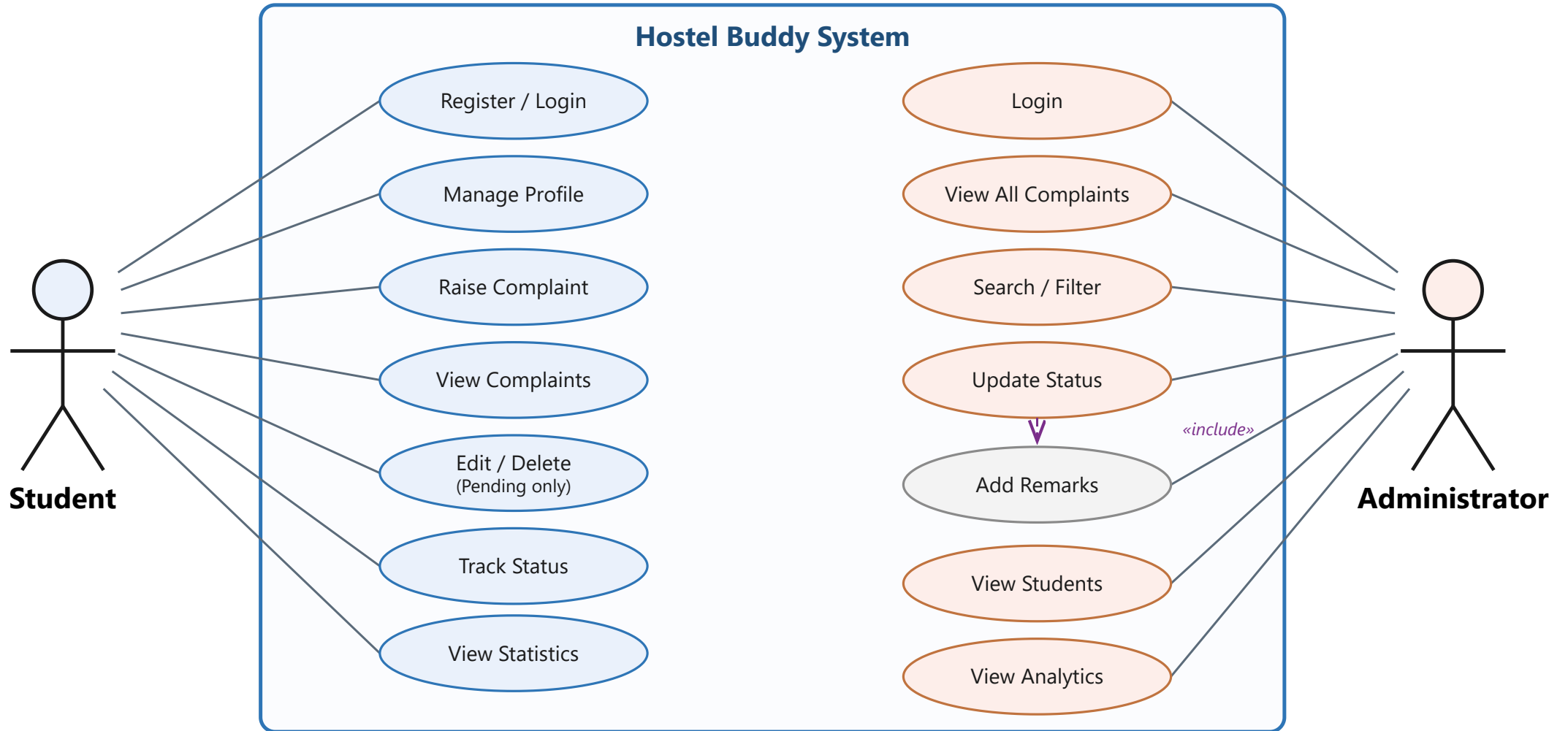
- Login
- View all complaints
- Search / filter complaints
- Update complaint status
- Add remarks
- View registered students
- View analytics

Non-Functional Requirements

- **Security** — hashed passwords, role-based access
- **Usability** — responsive, works on mobile
- **Maintainability** — independent modules
- **Reliability** — no complaint data loss

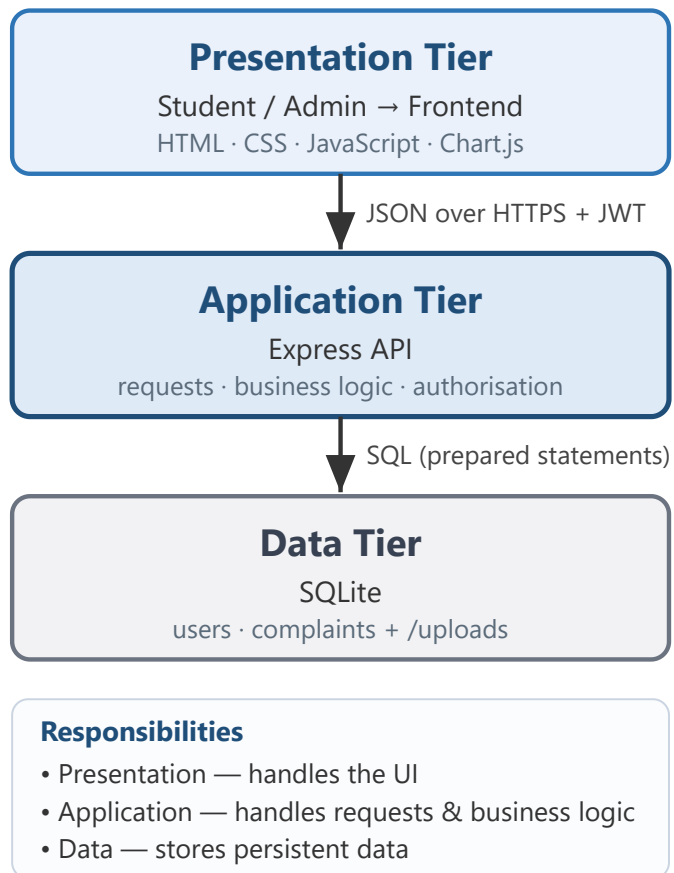
2 Use Case Diagram

2 min

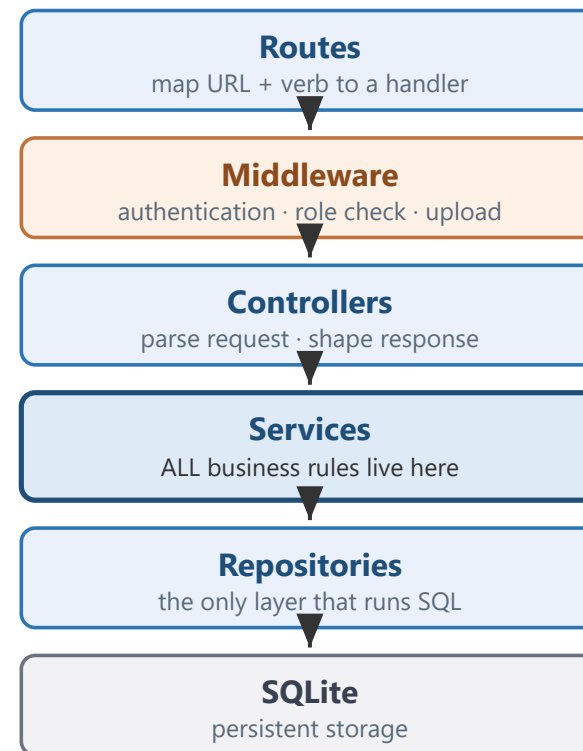


Two actors only. Administrator drives every status transition; the student is read-only on status.

Three-Tier View



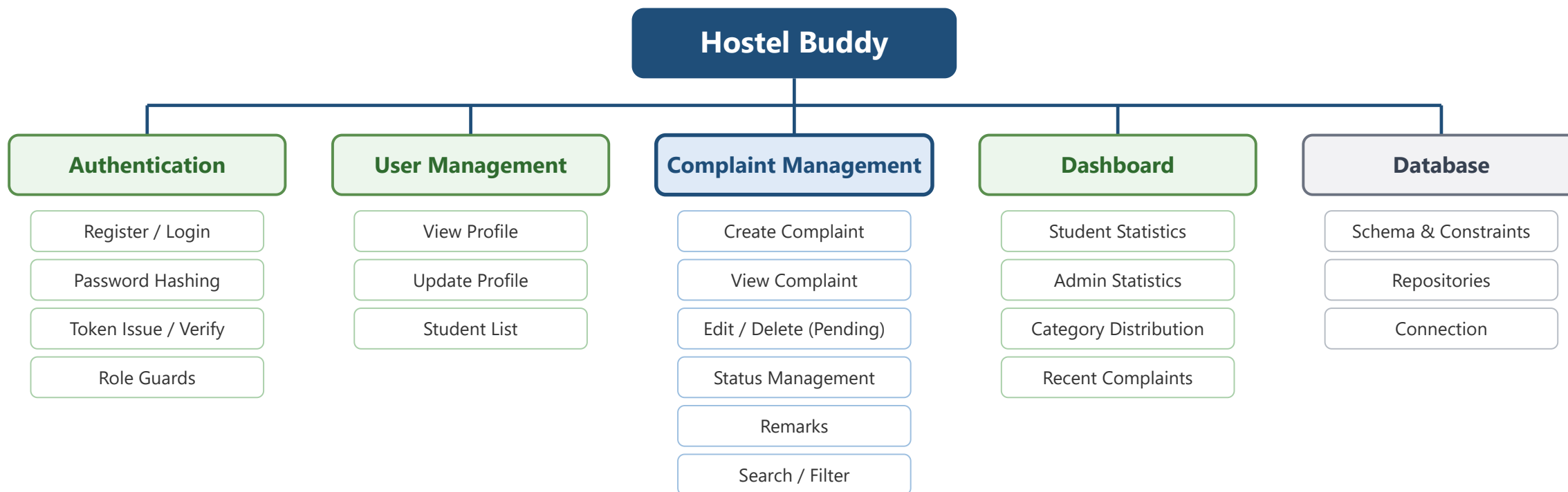
Inside the Application Tier



Each layer talks only to the layer directly below it.
Business rules never appear in the frontend or in SQL.

4 Module Decomposition

2–3 min



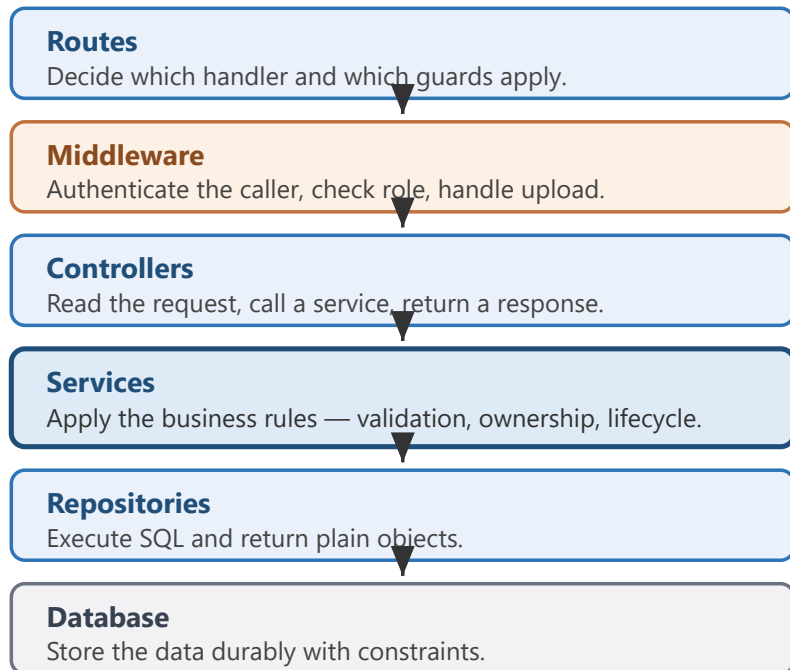
Purpose of decomposition

Each module has one specific responsibility — this makes the system easier to **maintain**, **test** and **extend**.

5 Layered Architecture & Data Flow

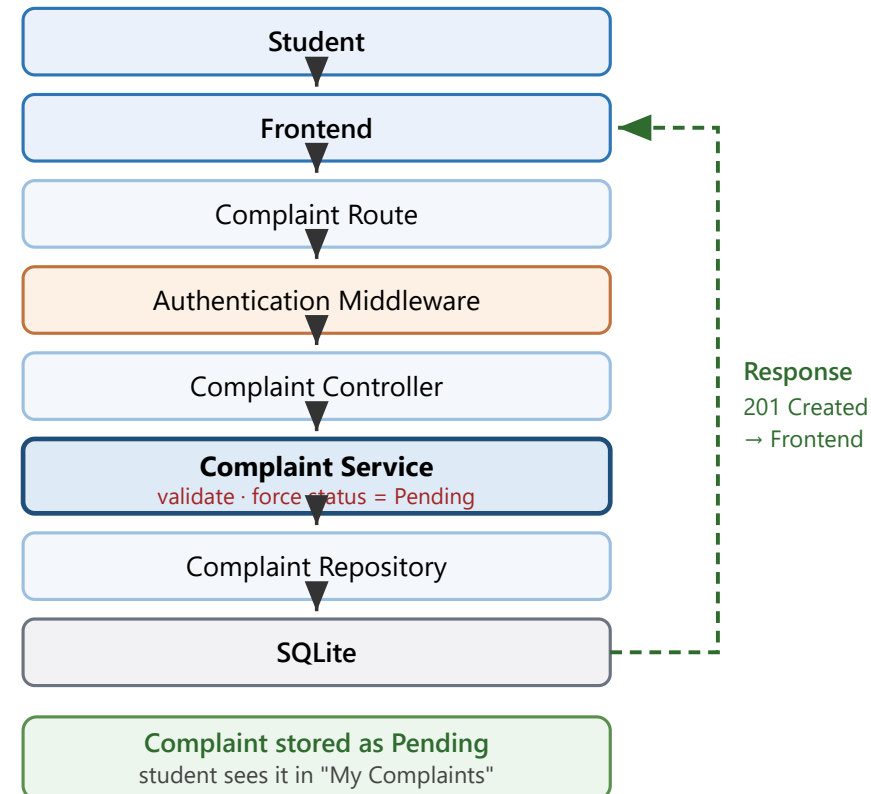
3 min

Layer Responsibilities



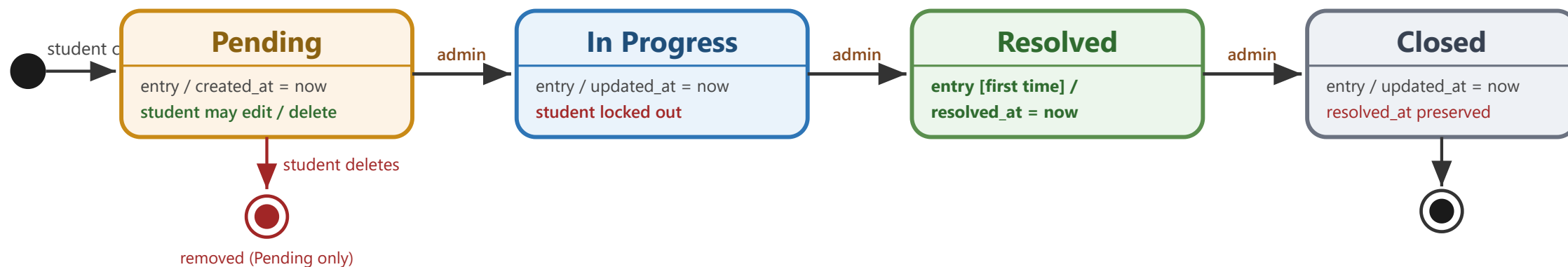
A request can never skip a layer.

Concrete Request — Raise Complaint



6 Complaint Lifecycle — State Diagram

2 min

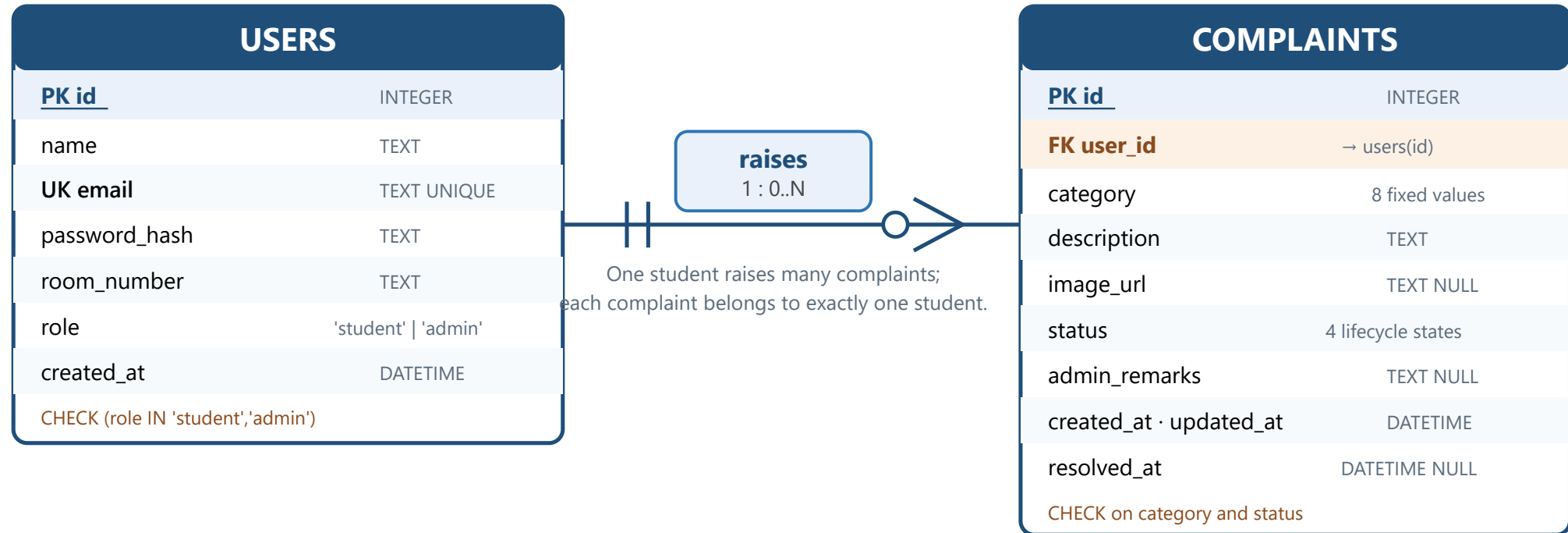


Lifecycle rules enforced by the system

1. The **student creates** the complaint — it always starts in **Pending**, never any other state.
2. The **administrator controls every status transition**; there is no student operation that changes status.
3. **Edit and delete exist only in Pending** — attempted later the server rejects it (409 NOT_PENDING).
4. **resolved_at is written once**, on first entry to Resolved — moving to Closed never overwrites it.

7 Database Design — ER Diagram

2 min



Two entities only. Admin is a **role** on USERS, not a separate table — authentication stays uniform. Categories and statuses are enforced by **CHECK constraints** rather than lookup tables — fixed sets, one fewer join.

Security

- **JWT authentication** — stateless signed tokens
- **bcrypt password hashing** — salted, never reversible
- **Role-based access control** — student vs administrator
- **Helmet / CSP** — security headers, scripts restricted to our origin
- **Rate limiting** on authentication endpoints

Why SQLite?

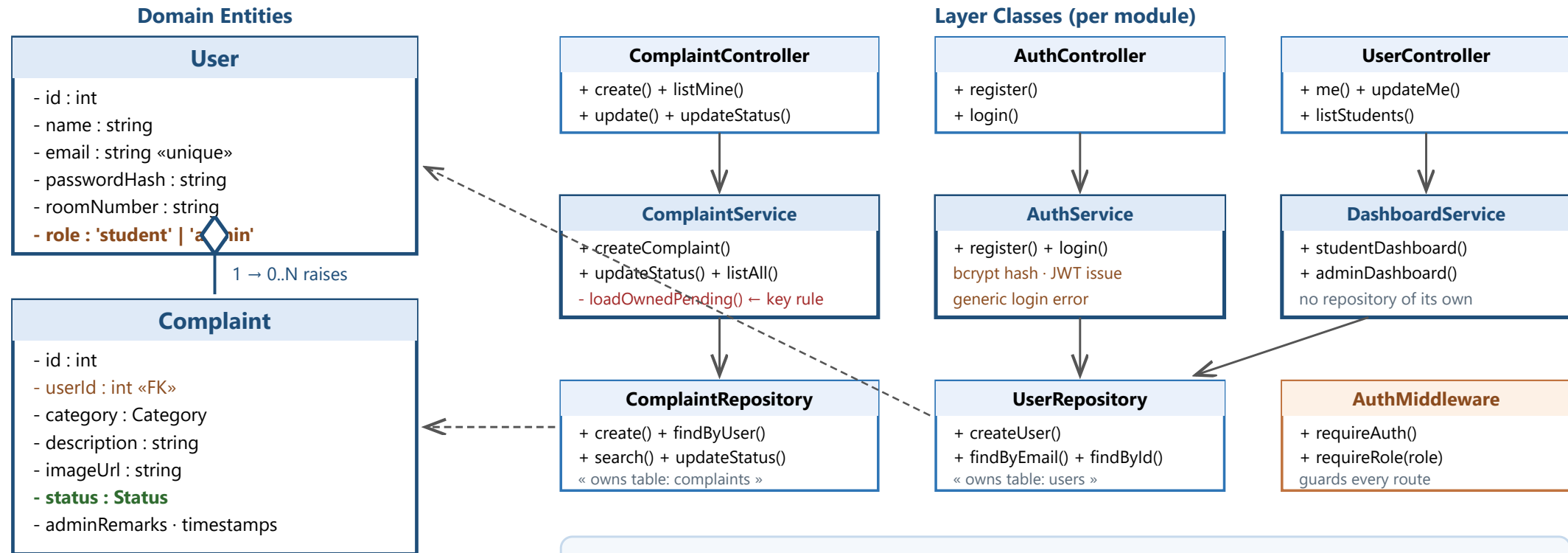
- Built into Node.js
- No compiler / native build requirement
- Simple setup for the model version

Why layered architecture?

- Separation of concerns
- Easier maintenance
- Easier testing
- Business logic independent from database access

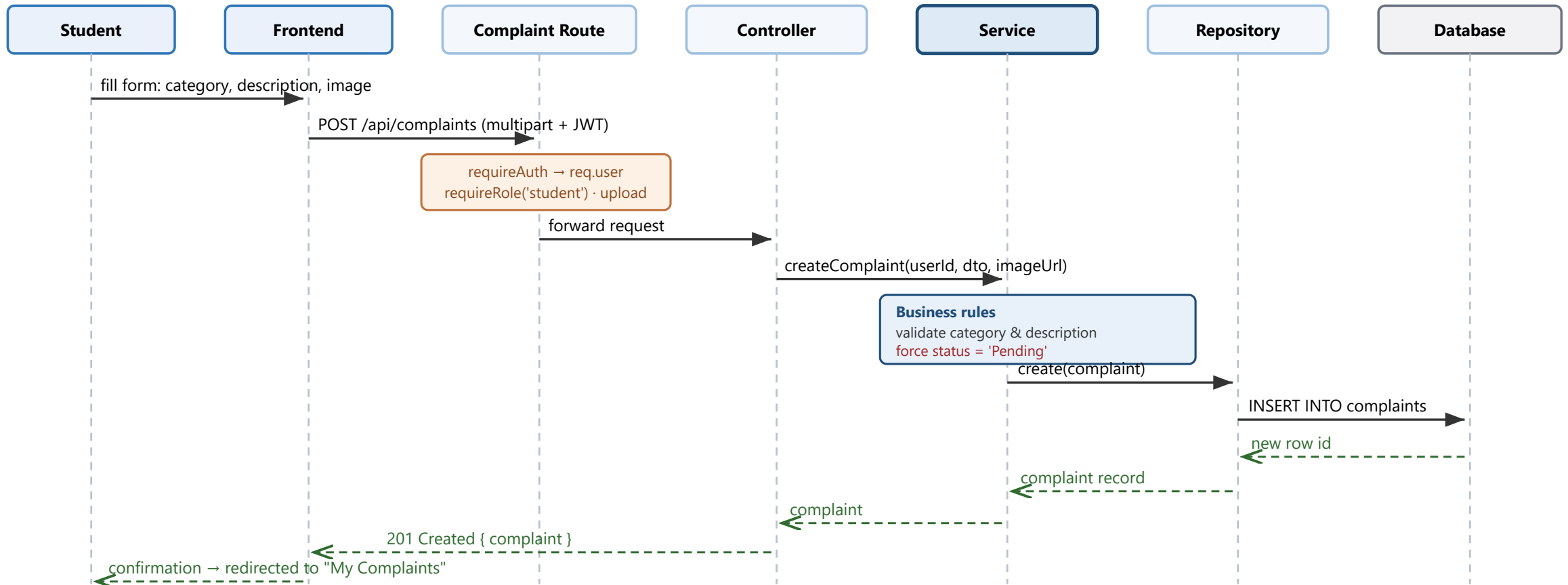
Why a Repository layer?

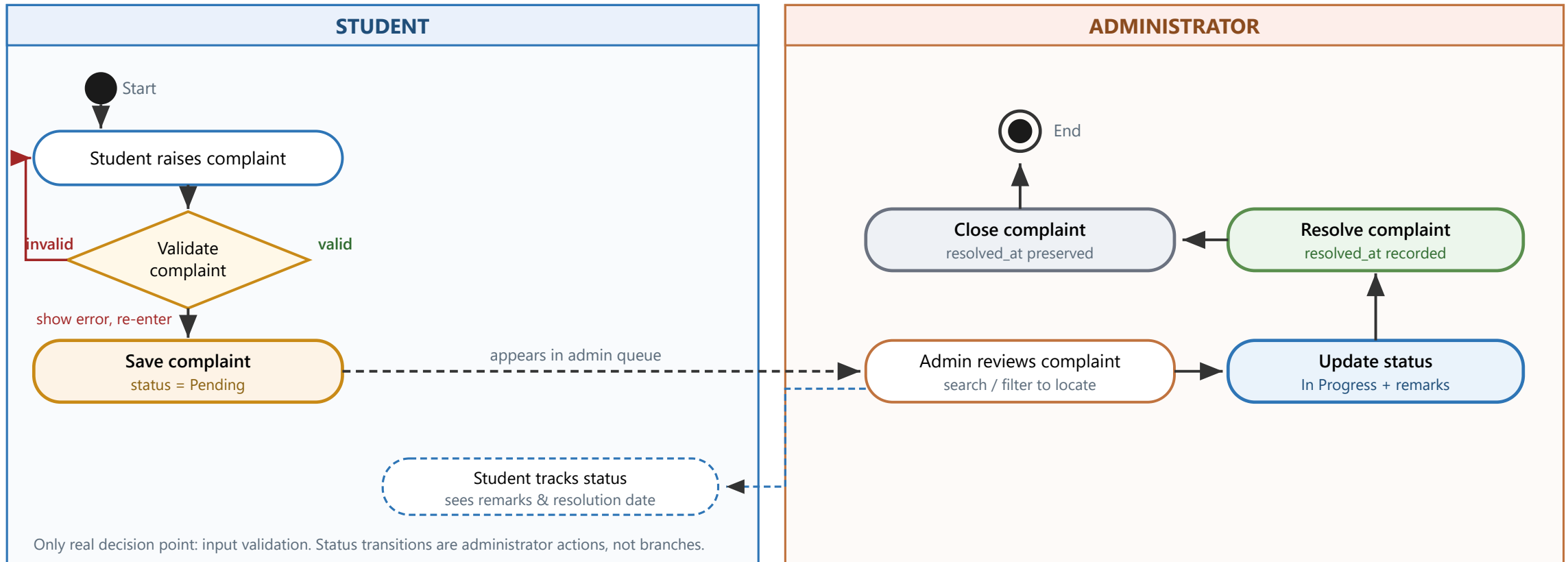
- Keeps SQL / database operations isolated
- Makes future database replacement easier

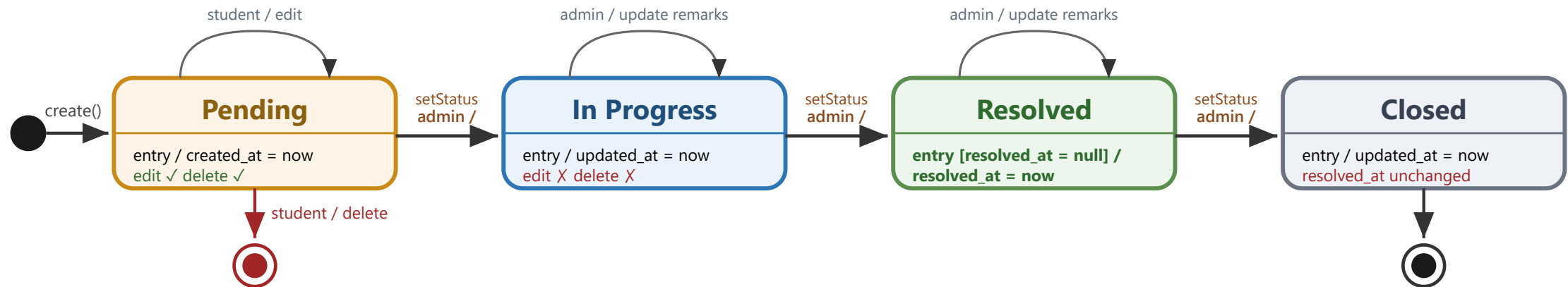


9 OOAD — B. Sequence Diagram: Raise Complaint

3 min (9A–9D)







Why the state machine matters architecturally

- The lifecycle is **forward-only** — a resolved complaint cannot be reopened in v1.0, which keeps resolution statistics unambiguous.
- Student permissions are attached to the **state**, not a separate flag — so permission can never fall out of step with status.
- **resolved_at is guarded on entry**, not assigned unconditionally — this is why Closed cannot falsify the resolution time.
- A state outside these four is **not representable** — a database CHECK constraint rejects it.

- 1 Hostel Buddy uses a **modular layered architecture** — four feature modules over a shared infrastructure layer.
- 2 **Business logic is separated** from presentation and database access — services hold the rules, repositories hold the SQL.
- 3 The **complaint lifecycle is controlled by authorization rules** — only the administrator advances status; students edit only while Pending.
- 4 The architecture is **maintainable and suitable for the current model version**, with a clear path to scale.

Thank you — questions?